

PROGRAMMAZIONE II

CLASSI E OGGETTI JAVA

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**

Dipartimento
di **INFORMATICA**

A.A. 2025/2026

CLASSE JAVA



PROGRAMMARE IN JAVA

- ▶ In Java, tutto è un oggetto
- ▶ Un oggetto è un valore di uno specifico tipo, definito da una classe

Per programmare in Java, è necessario definire classi!

15

METODO JAVA

I messaggi che possiamo inviare a un oggetto hanno una **signature**

returnType methodName(parameter₁ , ... , parameter_n)

dove i parametri sono nomi di variabili tipizzati: *parameterType parameterName*

Per esempio, il metodo `String getCarMainColor()`

- Non ha parametri, ma restituisce un intero

Mentre, il metodo `void paint(String mainColor, String topColor)`

- Ha due parametri, ma nessun valore di ritorno (è necessaria la parola chiave `void`)

METODO JAVA (CONT.)

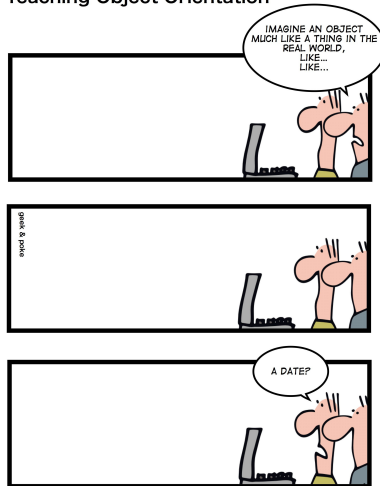
Overloading: una classe può avere diversi metodi con lo stesso nome

```
public class Car {
    String mainColor, topColor;

    void paint(String newColor) {
        mainColor = color;
        topColor = color;
    }
    void paint(String newMainColor, String newTopColor) {
        mainColor = newMainColor;
        topColor = newTopColor;
    }
}
```

ESEMPIO CLASS-ICO

Teaching Object Orientation



ESECUZIONE IN JAVA

Bello, ma

- ▶ Come possiamo eseguire una classe?
- ▶ Come possiamo inviare messaggi a un oggetto?

Una classe può essere eseguita solo se contiene un metodo speciale chiamato `main`

- ▶ In tal caso, l'esecuzione viene effettuata automaticamente dalla JVM

Per inviare un messaggio a un oggetto, bisogna

- ▶ Prima creare l'oggetto in memoria (da un altro oggetto)
- ▶ Poi chiamare uno dei suoi metodi

ENTRYPOINT DI UNA CLASSE JAVA

```
public class HelloWorld {  
    // Entrypoint della classe -- Questo metodo viene chiamato per avviare l'esecuzione  
    public static void main(String[] args) {  
        System.out.println("Hello_World!");  
    }  
}
```

Il parametro `args` è una lista di stringhe che rappresentano argomenti da riga di comando

► Nessun nome di classe/eseguibile viene passato tramite `args`

L'entrypoint deve essere `static`, il che significa che esiste un solo entrypoint

METODI STATICI IN JAVA

La parola chiave `static` rende un metodo legato alla classe anziché ai suoi oggetti

Un metodo statico

- ▶ Può essere chiamato anche se non è stato creato alcun oggetto della classe
- ▶ Non può accedere a contenuti specifici dell'oggetto (es. campi non statici)
- ▶ Può essere chiamato solo tramite il nome della classe: `HelloWorld.main()`

I metodi statici sono utili per eseguire azioni non correlate ad alcuna istanza di classe

- ▶ Classi di utilità o helper
- ▶ Calcoli privi di effetti collaterali

OGGETTI JAVA



ISTANZA DI CLASSE

Per usare effettivamente un oggetto bisogna allocare memoria e creare un riferimento

Questo viene fatto usando la parola chiave **new**

- ▶ Da un oggetto già istanziato: `Car myCar = new Car();`
- ▶ La JVM riserva automaticamente la memoria per un oggetto `Car`
- ▶ La JVM fornisce un riferimento al nuovo oggetto nella variabile `myCar`

Si ricordi che un riferimento è un nome di classe e una locazione di memoria (es. `Car@36c7a21b`)

ISTANZA DI CLASSE (CONT.)

Zero, uno o più riferimenti possono puntare allo stesso oggetto

```
Car car1, car2; 1
2
car1 = new Car(); // l'oggetto Car@36c7a21b viene creato 3
car1.paint("yellow") // imposta il colore a 'yellow' 4
// un riferimento all'auto gialla 5
6
car2 = car1; 7
// due riferimenti all'auto gialla 8
car2 = null; 9
// un riferimento all'auto gialla 10
car1 = null; 11
// di nuovo nessun riferimento all'auto gialla 12
13
// l'oggetto Car@36c7a21b viene liberato dal garbage collector 14
```

CREAZIONE DI OGGETTI

L'istruzione `var = new className()`

- ▶ Alloca la memoria nell'heap per un oggetto `className`
- ▶ Crea una nuova istanza della classe `className`
- ▶ Chiama il costruttore della classe `className`
- ▶ Restituisce un riferimento al nuovo oggetto `className` in `var`

// Il costruttore è un metodo speciale usato per inizializzare gli oggetti della classe

INTERAGIRE CON UN OGGETTO

Una volta istanziato, inviamo un messaggio a un oggetto chiamando uno dei suoi metodi

- Il metodo da chiamare non deve essere **static**

```
public class MainCar {  
  
    public static void main(String[] args) {  
        Car myCar = new Car()  
        myCar.paint("yellow") // imposta il colore a 'yellow'  
        System.out.println("Yellow car created!");  
    }  
  
}
```

1
2
3
4
5
6
7
8
9

METODI DI ISTANZA IN JAVA

Senza la parola chiave `static` un metodo sarà legato alle istanze di classe (oggetti)

Un metodo di istanza

- ▶ Può essere chiamato solo da oggetti istanziati
- ▶ Può accedere a contenuti specifici dell'oggetto e della classe (es. campi non statici e statici)
- ▶ Può essere chiamato solo tramite riferimento all'oggetto: `myCar.paint("yellow")`

 Da un contesto statico non si può chiamare un metodo non statico, mentre da un contesto non statico si può chiamare un metodo statico

METODI STATICI VS METODI DI ISTANZA

```
public class MainCar {  
  
    public static void main(String[] args) {  
        Car myCar = new Car()  
        myCar.paint("yellow") // metodo di istanza  
        System.out.println("Yellow car created!"); // metodo statico  
    }  
  
}
```

1
2
3
4
5
6
7
8
9

Viene chiamato il metodo statico `println()` del campo statico `out` della classe `System`

- `System` è una libreria standard fornita dalla JVM

METODI STATICI VS METODI DI ISTANZA (CONT.)

```
public classClazz {
    public static void main(String[] args) {
        staticPrint() // Ok -- Un contesto statico può chiamare un metodo statico
        Clazz myClazzInstance = new Clazz();
        myClazzInstance.print() // Ok -- Il metodo non statico viene chiamato
                                // da un oggetto (contesto non statico)
    }
    public static void staticPrint() {
        System.out.println("Static!");
    }
    public void print() {
        System.out.println("Non-static!"); // Ok -- Un metodo statico può essere
                                            // chiamato da un contesto non statico
    }
}
```

OGGETTO CORRENTE

All'interno di un metodo di istanza possiamo riferirci all'oggetto corrente con la parola chiave **this**

- ▶ Restituisce il riferimento dell'oggetto da cui il metodo è stato chiamato
- ▶ Non ha alcuna utilità nei contesti statici

```
public class Car {                                     1
    String color;                                     2
                                                    3
    public void paint(String newColor) {               4
        color = newColor;                             5
        this.printColor(); // L'oggetto corrente viene recuperato -- Il 6
    }                                                    // compilatore aggiunge implicitamente this. se omissso 7
                                                    8
    public void printColor() {                         9
        System.out.println(color);                    10
    }                                                  11
}                                                    12
```

CAMPI DI CLASSE

Anche ai campi è possibile accedere usando la notazione con il punto (dot-notation)

```
Car myCar = new Car(); 1
String carColor = my // il colore dell'auto è 'yellow' 2
myCar.color = "red"; // cambia il colore dell'auto in 'red' 3
```

Questo è in realtà sconsigliato, poiché viola il principio di information hiding

CAMPI DI CLASSE (CONT.)

Di nuovo, all'interno di un metodo di istanza possiamo riferirci all'oggetto corrente con `this`

```
public class Car {                                1
    String color;                                  2

    public void paint(String newColor) {           3
        this.color = newColor; // L'oggetto corrente viene recuperato -- Il 4
    } // compilatore aggiunge implicitamente this. se omissso 5
}                                                  6
                                                  7
```

RIFERIMENTO OBBLIGATORIO ALL'OGGETTO CORRENTE

In alcuni casi, la parola chiave `this` è obbligatoria

- Per distinguere i campi di classe dai parametri del metodo

```
public class Car {  
    String color;  
  
    public void paint(String color) {  
        this.color = color; // Ok -- Ora è chiaro che al campo di classe  
    }                       // viene assegnato il parametro del metodo  
}
```

1
2
3
4
5
6
7

TIPO RIFERIMENTO



RIFERIMENTI

Ogni classe `Clazz` genera un tipo riferimento `TClazz` parametrico su quella classe

- ▶ Ogni valore di `TClazz` è un riferimento a istanze di `Clazz`
- ▶ Ovvero, un puntatore a un'area di memoria contenente un oggetto `Clazz`

Ad eccezione dei valori primitivi, le variabili in Java possono contenere solo riferimenti

- ▶ Il valore speciale `null` appartiene a ogni tipo riferimento
- ▶ Tale valore può essere assegnato a tutte e sole le variabili che contengono riferimenti

```
Clazz clazz = null; // Ok  
String str = null; // Ok  
int i = null; // Errore di compilazione
```

1

2

3

RIFERIMENTI (CONT.)

Quando un riferimento ha valore `null`, non punta ad alcuna area di memoria

La notazione con il punto esegue automaticamente la dereferenziazione del puntatore

- ▶ `str.toString()` accederà alla memoria puntata da `str`
- ▶ Dereferenziare `null` genera quindi un'eccezione

Impostare un riferimento a `null` può risvegliare il garbage collector

STAMPARE UN RIFERIMENTO

Ogni oggetto in Java dispone del metodo `toString()` // vedremo più avanti il perché

- ▶ Questo metodo fornisce una rappresentazione in stringa di un oggetto
- ▶ Rappresenta una buona approssimazione di come appare un riferimento

```
Car myCar = new Car();
```

```
// l'output sarà qualcosa come 'Car@5acf9800'
```

1

2

3

STAMPARE UN RIFERIMENTO (CONT.)

Il metodo `toString()` viene chiamato implicitamente nei contesti in cui è attesa una stringa

```
String str = "Reference_=_ " + myCar;  
System.out.println(str); // anche qui toString() viene chiamato implicitamente  
// l'output sarà qualcosa come 'Reference = Car@5acf9800'
```

1

3

Perché `str.toString()` non produce un output del tipo `String@4617c264` ?



Con l'**overriding** è possibile modificare l'implementazione di un metodo

DOMANDE E RISPOSTE

